

Network Operations Strategy Memo

Graph-Based Delivery Intelligence for the Delhivery Logistics Network

To: Head of Network Operations **From:** Data Science Team **Re:** ETA accuracy & bottleneck remediation **Analysis window:** 12 Sep – 6 Oct 2018 · 14,817 trips · 26,369 corridor legs · 1,657 facilities

Executive summary

OSRM under-estimates delivery time on **94.7% of legs** — actual time runs at a median of **2.0× the OSRM estimate**, so the gap is structural, not occasional. Treating the network as a connected graph (facilities as nodes, corridors as edges) rather than a set of independent point-to-point estimates reveals that delay is **highly concentrated**: just **3 hubs handle 39% of all excess delay in the network while touching only 17.5% of legs**. Fixing those three hubs is the single highest-leverage move available. A graph-enhanced ETA model already cuts prediction error by nearly **12%** over a strong baseline, and a route-type policy shift on long-haul corridors recovers further SLA headroom.

The three actions below are ranked by return on effort.

1. The top 5 bottleneck hubs

Hubs are ranked on a composite of **structural centrality** (share of shortest paths through the hub), **delay contributed**, and **throughput** — a hub matters only when it is central *and* slow *and* busy. Betweenness is the share of network shortest paths that pass through the facility.

Rank	Hub	Betweenness	Throughput (legs)	Excess delay handled	Read
1	Gurgaon_Bilaspur_HB (Haryana)	0.22	1,991	~697,000 min	On 22% of all shortest paths — the network's single point of failure
2	Bangalore_Nelmangala_H (Karnataka)	0.13	1,433	~353,000 min	Southern gateway; high volume + high delay
3	Bhiwandi_Mankoli_HB (Maharashtra)	0.07	1,404	~310,000 min	Mumbai gateway; busiest western node
4	Hyderabad_Shamshabad_H (Telangana)	0.09	734	~161,000 min	Central-south chokepoint
5	Kolkata_Dankuni_HB (West Bengal)	0.09	481	~208,000 min	Eastern gateway; worst delay-per-leg of the five

The concentration finding: the **top 3 hubs carry 39.2%** of all network excess delay across only 17.5% of legs; the **top 5 carry 46.8%**. This is a Pareto network — a handful of upgrades move the whole SLA curve.

2. Corridor-specific interventions

The chronic-delay audit (corridors running >1.2× OSRM with real volume) points to specific trunk routes, almost all anchored on Gurgaon_Bilaspur:

Corridor	Delay vs OSRM	Recommended intervention
Gurgaon → Bangalore	1.85×	Parallel-route / scheduled FTL lane — highest single-corridor delay contributor
Gurgaon → Kolkata	2.03×	Add intermediate cross-dock to break the long unbuffered leg
Guwahati → Delhi	2.53×	Facility-process upgrade at origin — worst ratio; North-East dwell is the issue
Gurgaon → Hyderabad / Bhiwandi	~1.9×	Route-type shift to FTL (see §4); both are long-haul where FTL pays off

Intervention logic: **parallel route** where one corridor is saturated, **facility upgrade** where the delay is dwell (origin processing) rather than transit, **route-type shift** where a cheaper-but-slower mode is being used on distance bands that favour FTL.

3. Smarter ETA: the graph model already pays off

A graph-enhanced model (baseline trip features **plus** node2vec embeddings and centrality of the source/destination facilities) was benchmarked against a strong gradient-boosted baseline on a leakage-free, trip-grouped hold-out:

Model	Mean error (MAE)	Within 15% of actual
Baseline	27.5 min	51.7%
Graph-enhanced	24.3 min	55.4%
Gain	-11.6% error	+3.7 points

The graph signal — *where a facility sits in the network* — carries information the baseline cannot see. Roll this model into the promise-time engine; the accuracy gain directly reduces over- and under-promising.

4. FTL vs Carting policy

Counterfactual modelling (predicting each leg's time under both route types) gives a clean rule keyed to distance:

- **Long-haul (200-800 km): default to FTL.** It saves **60-77 minutes per shipment**, and the time value clears the cost premium with room to spare (breakeven $\approx$ Rs 6-7 per saved minute).
- **Short / mid-haul (<200 km): default to Carting.** FTL's time saving is small (8-20 min) and does not justify the premium.
- **Use the source hub's graph position as a tie-breaker:** legs originating at high-betweenness hubs should bias toward FTL, because delay there propagates network-wide.

This is a tunable framework, not a fixed verdict — the cost premium and SLA value are explicit levers operations can set.

5. Quantified impact — if the top 3 hubs are upgraded

Conservative, clearly-flagged assumptions: 30% of excess delay at an upgraded hub is addressable; each delayed shipment carries an illustrative Rs 120 SLA/churn cost.

- **Late-delivery reduction:** the top-3 hubs sit on 4,500+ breaching legs; a 30% remediation removes delay from **~1,350 shipments in the 24-day window** and shifts the network-wide breach rate measurably, since these hubs gate ~39% of all excess delay.
- **Revenue-at-risk recovered:** $\approx$ **Rs 1.6 lakh in the analysis window**, scaling to $\approx$ **Rs 25 lakh annually** on this traffic slice alone — before counting the retention value of more reliable ETAs.
- **Where to spend first:** Gurgaon_Bilaspur. At 22% betweenness it is both the largest delay contributor and the largest structural risk; any rupee of capacity or process investment there has the widest blast radius.

Recommended next steps

1. **Commission a dwell-time audit at Gurgaon_Bilaspur** — separate transit delay from facility processing delay to size the fix.
2. **Pilot a scheduled FTL lane on Gurgaon → Bangalore** and measure the SLA delta against the model's prediction.
3. **Deploy the graph-enhanced ETA model** into promise-time, starting with the top-20 corridors.
4. **Adopt the distance-keyed FTL/Carting rule** and instrument the realized cost-time trade-off.

Supporting figures: network map ([fig_network.png](#)), hub ranking ([fig_bottleneck_hubs.png](#)), chronic corridors ([fig_delay_corridors.png](#)), centrality-vs-delay ([fig_betw_vs_delay.png](#)), model benchmark ([fig_model_compare.png](#)), route-type trade-off ([fig_ftl_carting.png](#)). All numbers reproducible via the `src/` pipeline.